CS401 MPP

W3D3 Overview

The Stream API (part 1)

Overview of Today and Tomorrow

- Intro to Java 8 Streams
- Streams Basic Facts
- Ways to Create Streams
- Intermediate Operations
- Stateful Intermediate Operations
- Terminal Operations
- Working with Optional
- Reduce
- Collecting Results
- Primitive Type Streams
- Lambda Libraries

Wholeness Statement

- The stream API is an abstraction of collections that supports aggregate operations like filter and map.
  - These operations make it possible to process collections in a declarative style that supports parallelization, compact and readable code, and processing without side effects.
- Deeper laws of nature are ultimately responsible for how things appear in the world.
  - Efforts to modify the world from the surface level lead to struggle and partial success.
  - Affecting the world by accessing the deep underlying laws that structure everything can produce enormous impact with little effort.
  - The key to accessing and winning support from deeper laws is going beyond the surface of awareness to the depths within.

CS401 MPP

The Stream API

Intro to Java 8 Streams

What are Streams

- A stream is a way of representing data in a collection (and in a few other data structures) which supports functional-style operations to manipulate the data.
- From the API docs: A stream is “a sequence of elements supporting sequential and parallel aggregate operations.”
- Streams provide new ways of accessing and extracting data from Collections.

Why Use Streams?

- To understand why they are used, consider the following problem:
  - Given a list of words (say from a book), count how many of the words have length > 12.

```
// imperative-style solution
int count = 0;
for(String word : list) {
     if(word.length() > 12)
         count++;
}
```

Issues:
I. Relies upon shared variable count, so is not threadsafe
II. Commits to a particular sequence of steps for iteration
III. Emphasis is on how to obtain the result, not what is needed

Functional-Style Solution

```
final long count
     = list.stream()
       .filter(w -> w.length() > 12)
       .count();
```

Advantages:

I. Purely functional, so threadsafe

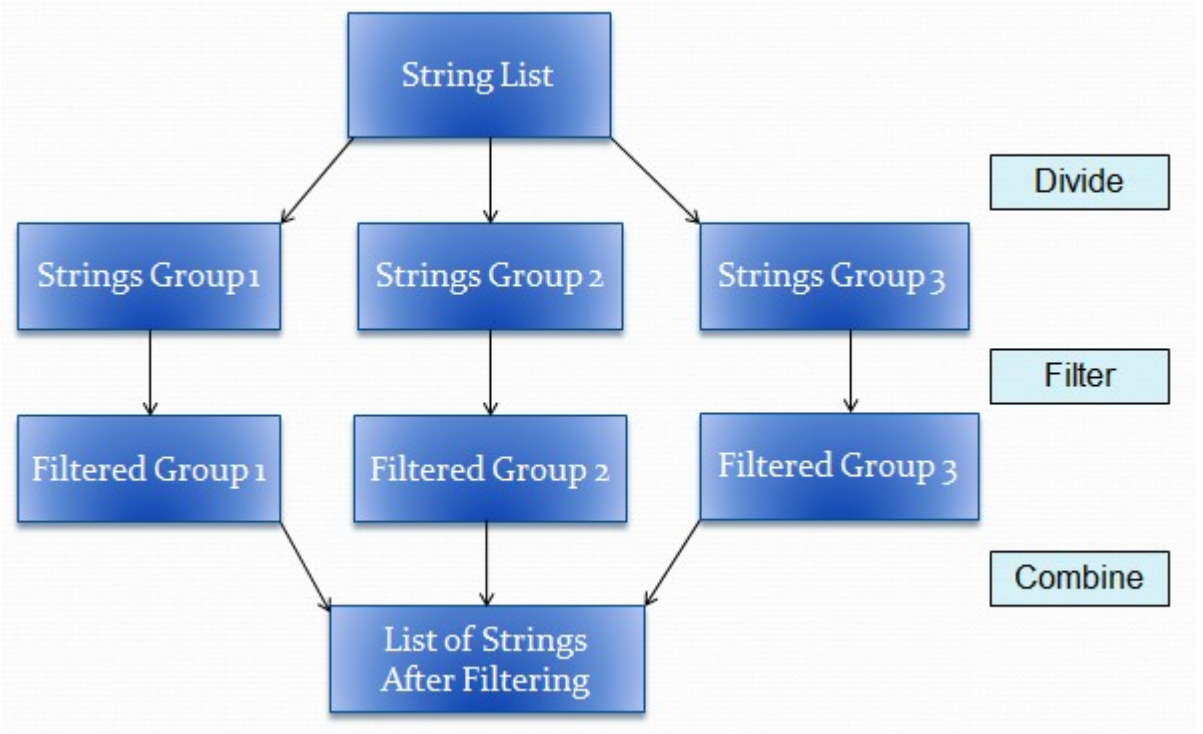
II. Makes no commitment to an iteration path, so more parallelizable

III. Declarative style – “what, not how”

IV. With Java 8 it is easy to transform into a parallel processing solution

Parallel-Processing Solution

```
final long count
     = list.parallelStream()
       .filter(w -> w.length() > 12)
       .count();
```



CS401 MPP

The Stream API

Streams: Basic Facts

Facts About Streams

1. Streams do not store the elements they operate on
   - Typically they are stored in an underlying collection, or they may be generated on demand.

2. Stream operations do not mutate their source
   - Instead, they return new streams that hold the result.

3. Java Implementation:
   - The methods on the Stream interface are implemented by the class ReferencePipeline.
   - The implementation involves a combination of technical operations internal to the stream package

Facts About Streams

4. Stream operations are lazy whenever possible
   - Elements in the stream are accessed in a way analogous to streaming video: only the elements that are absolutely necessary are accessed, and the intermediate operations act on those
   - When more elements are needed, they are accessed

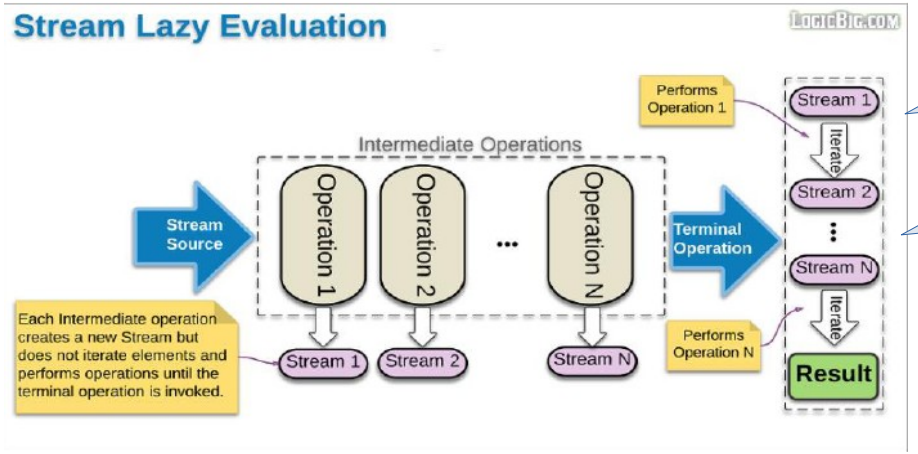
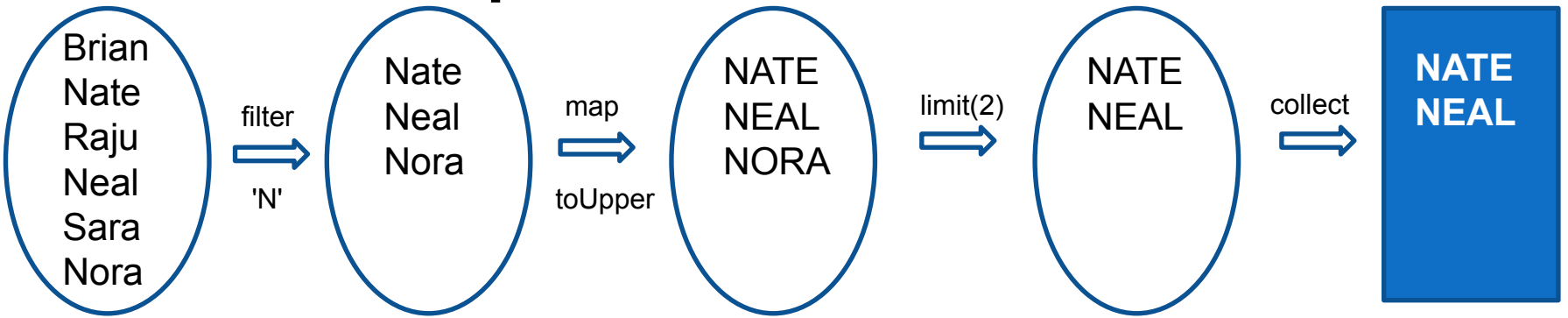


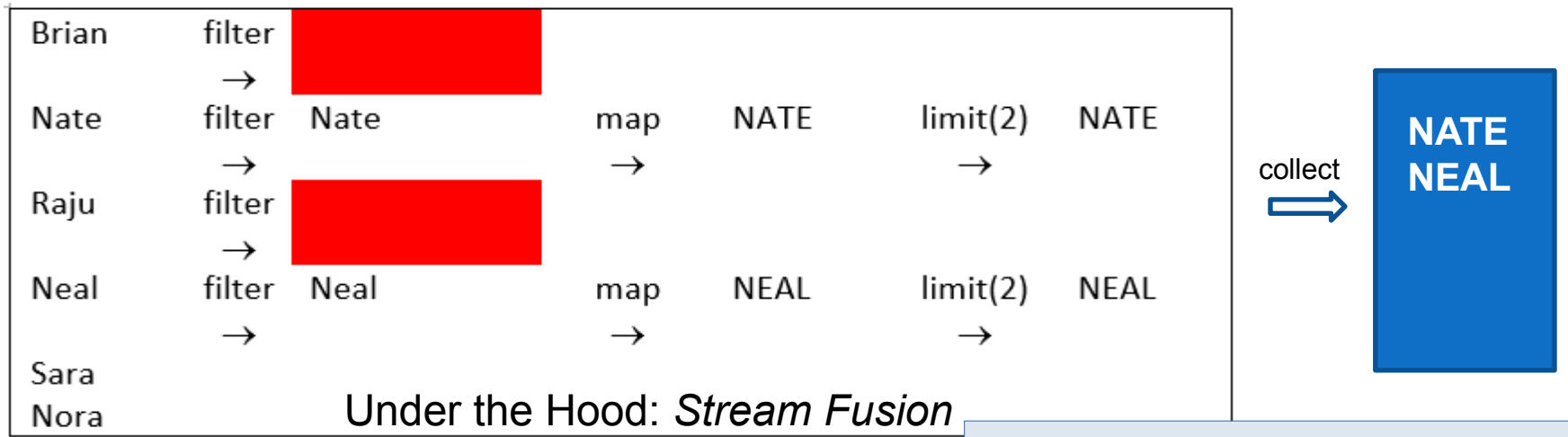
Diagram shows **User View** of streams

See: lesson9.lecture.lazystream

Stream Pipelines: Under the Hood



User View: New Stream after Each Operation



Under the Hood: *Stream Fusion*

Loops fused to produce single stream

See: lesson9.lecture.lazystream.LoopFusion.java

Template for Using Streams

1. Create a stream
   - Typically, the stream is obtained from some kind of Collection, but streams can also be generated from scratch
2. Create a pipeline of operations
   - Each of the operations performs some stream transformation and returns a new stream, these are called intermediate operations
3. End with a terminal operation
   - The terminal operation produces a result
   - It also forces lazy execution of the operations that precede it

Example

- In this example from lesson 8 we clearly see the 3 steps described on the previous slide

```
List<String> startsWithLetter =
   list.stream()                          //1. create the stream
       .filter(name -> name.startsWith(letter))  //2. build pipeline
       .collect(Collectors.toList()); //3. invoke terminal operation
```

CS401 MPP

The Stream API

Ways to Create Streams

Ways to Create Streams

1. Obtain a **`Stream`** from any **`Collection`** object with a call to **`.stream()`**
   - This default method was added to Collection interface in Java 8
2. Get a **`Stream`** from an array of objects using the static ***`of`*** method
   - This does not work correctly with primitive types
   - Can use Object wrappers for primitives

```
Integer[] arrOfInt = {1, 3, 5, 7};
Stream<Integer> strOfInt = Stream.of(arrOfInt);
```

Works as expected

```
int[] arrOfInt = {1, 3, 5, 7};
Stream<Integer> strOfInt = Stream.of(arrOfInt);
```

Becomes a 1 element stream containing only the `int[]`

Ways to Create Streams

3. Get a Stream from any sequence of arguments
   - The ***of*** method accepts a varargs argument

```
Stream<String> song
   = Stream.of("gently", "down", "the", "stream");
```

For a review of varargs see:
https://docs.oracle.com/javase/1.5.0/docs/gu

4. Two ways to obtain *infinite* streams:
   a) Generate
   b) Iterate

Remember: stream operations are lazy

Generate

- The **`generate`** function accepts a **`Supplier<T>`**
  - In practice, this means that it accepts functions (lambda expressions) with zero parameters

```
interface Supplier<T> {
     T get();
}
```

```
Stream<String> echoes = Stream.generate(() -> "Echo");
```

Example:
Stream of constant values (“Echo”)

```
Stream<Double> echoes = Stream.generate(Math::random);
```

Example:
Stream of random numbers

Iterate

- The **`iterate`** function accepts a seed value (of type T) and a UnaryOperator<T> argument

```
interface UnaryOperator<T> {
     T apply(T t);
}
```

```
Stream<Integer> stream2 = Stream.iterate(1, n -> n + 1);
```

Example:
Stream of natural numbers
Here **T** is **`Integer`**

You cannot use **`int`** in place of **`Integer`**.

We will discuss streams of primitives later in the lesson

See: lesson9.lecture.iterate

Extracting Substreams

- **`stream.limit(n)`** returns a new Stream that ends after **`n`** elements
  - Or when the original Stream ends if it is shorter
  - Useful for cutting infinite streams down to size

```
Stream<Double> randoms
  =  Stream.generate(Math::random).limit(100);
```

Stream with 100 random numbers

- **`stream.skip(n)`** discards the first n elements

Combining Streams

- **`Stream.concat(Stream, Stream)`** concatenates two streams
  - Concat() is a static method of the Stream class

```
Stream<Character> combined =
  Stream.concat(characterStream("Hello"), characterStream("World"));
```

Produces the Stream :
['H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd']

```
public static Stream<Character> characterStream(String s) {
     List<Character> result = new ArrayList<>();
     for (char c : s.toCharArray())
          result.add(c);
     return result.stream();
}
```

This is the characterStream method used in the above example

Exercise 9.1

1. Use **`Stream`**'s iterate method to produce a **`Stream`** consisting of all the odd natural numbers 1, 3, 5, . . .
2. Modify your solution so that your **`Stream`** consists of exactly these numbers: 9, 11, 13, 15
   - Print your Stream to the console (somehow)

Main Point 1

- The iterate operation on a Stream makes it possible to generate an infinite sequence of values based on two pieces of data: The initial value, and a rule or principle that tells how one value should be transformed to the next.
- The iterate method is an expression of the Principle of Diving.
  - The "correct angle" is the initial value
  - "Letting go" has the right effect because there is an underlying rule that directs flow from the initial value to subsequent values
  - During meditation, what guides awareness from one level to the next – the underlying principle or rule – is the gentle pull to move toward what is most charming at each moment while awareness remains lively and awake

CS401 MPP

The Stream API

Intermediate Operations

Filter

- Use ***filter*** to extract a Substream
  - Of the elements that satisfy the specified criteria
  - ***filter*** accepts as its argument a Predicate<T>

```
interface Predicate<T> {
    boolean test(T t);
}
```

  - ***filter*** returns a Stream, so filters can be chained

```
words.stream()
  .filter(name -> name.contains("" + c))
  .filter(name -> !name.contains("" + d))
  .filter(name -> name.length() == len)
  .count();
```

Map

- Use ***map*** to transform each element
  - ***map*** accepts a **Function** interface
  -
```
interface Function<T,R> {
     R apply(T t);
}
```
  - ***map*** accepts a **Function** as input and returns a **Stream<R>** (where **R** is the return type of Function)
    - ***map*** can therefore be chained

```
List<Integer> list = ...
List<String> stringsFromIntegers
     = list.stream().map(x -> x.toString())
               .collect(Collectors.toList());
```

Given a List<Integer> creates a List<String> where each item is the string representation of that Integer

map with Constructor Method Refs

- **`Class::new`** is a fourth type of method reference, where the method is the new operator.
  - Typical translation: **`z -> new Class(z)`**
  - Compiler must select which constructor
  - Compiler determines this based on the context

```
List<String> labels = ...
Stream<Button> stream = labels.stream().map(Button::new);
List<Button> buttons = stream.collect(Collectors.toList());
```

In this context **`Button::new`**
the compiler clearly selects:
`str -> new Button(str)`

Which realizes a **`Function`** Interface
as is required by **`map`**

Another Example

```
public class StringCreator {
    public static void main(String[] args) {
      Function<char[], String> myFunc = String::new;
      char[] charArray =
              {'s','p','e','a','k','i','n','g','c','s'};
      System.out.println(myFunc.apply(charArray));
      System.out.println(new String(charArray));
    }

}
```

In this case, **`String::new`** is short for the lambda expression **`charArray -> new String(charArray)`**

Which, as required, is a realization of the Function interface.

The output for both is:
speakingcs

See: lesson9.lecture.newstring

Array Constructor

- **`int[]::new`** is another constructor reference
  - short for the lambda: **`len -> new int[len]`**
  - where **`len`** is an integer that is used as the new array length

The toArray method

- Problem:
  - Turn a **`Stream<String>`** into a **`String[]`**

```
String[] vals = stringStream.toArray(); //compiler error
```

  - Solution:

```
String[] vals = stringStream.toArray(String[]::new);
```

Use a constructor reference to specify the correct type!

See: lesson9.lecture.constructorref.GenericArray

FlatMap

- Transforms each element into a Stream
  - In essence making a Stream of Streams
  - Then ‘flattens’ the result back into a single Stream

```
List<String> list = Arrays.asList("Joe", "Tom", "Abe");
Stream<Stream<Character>> result
       = list.stream().map(s -> characterStream(s));

// result is:  [['J', 'o', 'e'],['T', 'o', 'm'],['A', 'b', 'e']].
```

Using the characterStream method we defined earlier

Map does not flatten

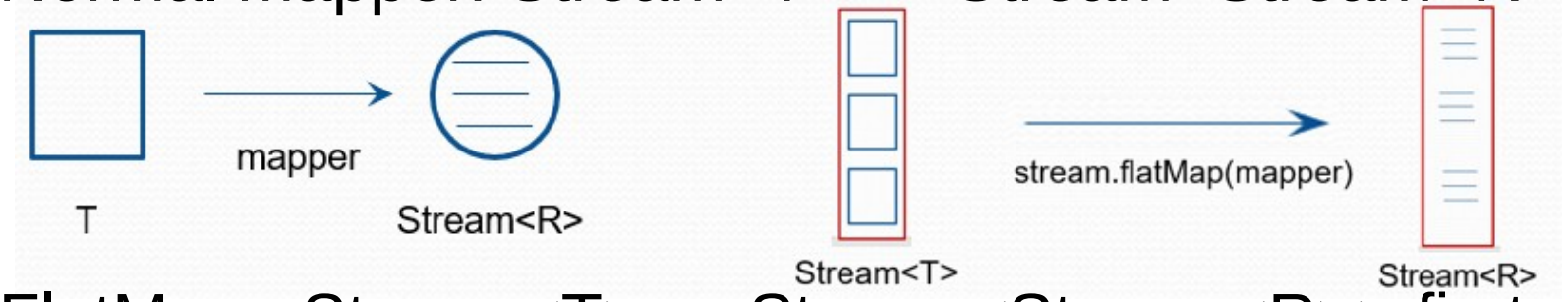
```
List<String> list = Arrays.asList("Joe", "Tom", "Abe");
Stream<Character> result
       = list.stream().flatMap(s -> characterStream(s));

// result is:  ['J', 'o', 'e', 'T', 'o', 'm', 'A', 'b', 'e'].
```

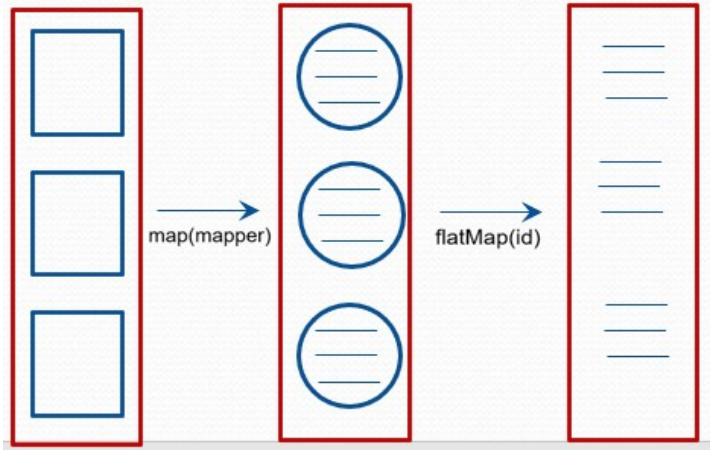
flatMap flattens the result into a single Stream

Visually

- Normal mapper: Stream<T> → Stream<Stream<R>>



- FlatMap: Stream<T> → Stream<Stream<R>> first
  - Then Stream<Stream<R>> → Stream<R>



Exercise 9.2

- What is the following code doing?
  - What is the output when it is run?

```
public static void main(String[] args) {
    List<Integer> ints = Arrays.asList(3,5,2,3,8);
    List<int[]> intArrs = ints.stream()
                              .map(int[]::new)
                              .collect(Collectors.toList());
    List<String> intArrsStr = intArrs.stream()
                                     .map(Arrays::toString)
                                     .collect(Collectors.toList());
    System.out.println(intArrsStr);
}
```

See: lesson9.lecture.constructorref.IntArrayExample

Main Point 2

- Java 8 makes it possible to dynamically construct objects using lambdas or method references
  - An application of this feature is that a developer can construct an entire Stream of new objects with very little code, by combining map with a constructor method reference
- This combination of new Java features reflects a deeper quality that is found in the field of intelligence itself:
  - Intelligence naturally tends toward creative expression.
  - Contact with our own deepest levels of intelligence through transcending results in enhanced ability for creative expression

CS401 MPP

The Stream API

Stateful Intermediate Operations

Stateful

- Most transformations discussed so far:
  - `map`, `filter`, `limit`, `skip`, `concat` – have been stateless
  - Each element of the stream is processed and forgotten
- Two stateful transformations seen so far:
  - `limit` and `skip`.

Recall the "under the hood" view of Streams.

- Two other stateful transformations available:
  - `distinct` and `sorted`.

Example of distinct

```
Stream<String> uniqueWords
  = Stream.of("merrily", "merrily", "merrily","gently").distinct();
// output: ["merrily", "gently"]
```

Sorted Example

Sorted accepts a **Comparator**

```
//sort by decreasing lengths of words
List<String> words
   = Arrays.asList("Tom", "Joseph", "Richard");
Stream<String> longestFirst
   = words.stream()
          .sorted((String x, String y) -> y.length()-x.length());
System.out.println(longestFirst.collect(Collectors.toList()));
//output: Richard, Joseph, Tom
```

See: lesson9.lecture.comparators1

Note: Sorting logic: x comes before y if y.length() – x.length() is negative, which happens when x is longer. So "Richard" comes before "Tom"

Note: distinct() and sorted() prevent stream fusion because they are stateful, so pipelines that use these are slightly less efficient

Note: This code uses some functional techniques, but notice that the Comparator still has the flavor of “how” rather than “what”.

More Functional Comparators

- In the previous example, we wanted to sort “by String length”, in reverse order
  - Rather than specifying how to do that, we can use the new static comparing method in Comparator:

```
Stream<String> longestFirst
  = words.stream().sorted(Comparator.comparing(String::length).reversed());
```

See: lesson9.lecture.comparators1

Comparator.comparing()

- **`Comparator.comparing()`** takes a **`Function<T,U>`** argument and returns another **`Comparator`**
- The type **`T`** is of what is being compared
  - In our example **`T`** was a **`String`**
- The type **`U`** is the type that will actually be compared
  - The type of **`U`** was **`Integer`** in our example
  - Since we are comparing lengths of words

More Intuitive

```
Function<String, Integer> byLength = x -> x.length();
Stream<String> longestFirst
     = words.stream().sorted(Comparator.comparing(byLength).reversed());
```

Note: **`reversed()`** is a default method in **`Comparator`**
that reverses the order defined by the instance of **`Comparator`** that it is being applied to.

Note: The lambda **`x -> x.length()`** is the same as **`String::length`**

`Comparing` Examples

- Create two **`Comparator<Employee>`**
  - One that compares **`Employee`**s by name
  - Another that compares by salary

```
Comparator<Employee> NameComparator
  = Comparator.comparing(Employee::getName);
```

```
Comparator<Employee> SalaryComparator
  = Comparator.comparing(Employee::getSalary);
```

Comparators:Consistent with Equals

Recall consistency with equals issue in Lab 8

- Remember that to sort Employees by name
  - Where an Employee has a name and a salary
  - We needed to also consider the salary
    - or else the Comparator is not consistent with equals
- The following is not consistent with equals:

```
Collections.sort(emps, (e1,e2) -> {
      if(method == SortMethod.BYNAME) {
            return e1.name.compareTo(e2.name);
      } else {
            if(e1.salary == e2.salary) return 0;
            else if(e1.salary < e2.salary) return -1;
            else return 1;
      }
});
```

See: lesson9.lecture.comparators2

The Fix

- This approach is “how”-oriented
  - And more declarative by using the **`comparing`** and **`thenComparing`** methods of Comparator.
  - At the same time, we make it consistent with **`equals`**

```
Function<Employee, String> byName = e -> e.getName();
Function<Employee, Integer> bySalary = e -> e.getSalary();

public void sort(List<Employee> emps, final SortMethod method) {
    if(method == SortMethod.BYNAME) {
        Collections.sort(emps, Comparator.comparing(byName).thenComparing(bySalary));
    } else {
        Collections.sort(emps, Comparator.comparing(bySalary).thenComparing(byName));
    }
}
```

Notes on **comparing** and **thenComparing**

- comparing is a static method of Comparator
  - Returns something that implements Comparator
  - But Remember: implementations do not inherit static from interfaces
  - Therefore it cannot be chained
- thenComparing is a default method on Comparator
  - Also returns an implementation of Comparator
  - Default methods do get inherited
  - Therefore it can be chained

It adds an additional compare method after the given Comparator which executes only when the compare method inside the original Comparator returns 0

Exercise 9.3

- Use the **`comparing`** and **`thenComparing`** methods of Comparator to sort a list of Accounts first by balance, then by ownerName
  - See lesson9.exercise_3 in the InClassExercises

```
public class Account {
    private String ownerName;
    private int balance;
    private int acctId;
    public Account(String owner, int bal, int id) {
        ownerName = owner;
        balance = bal;
        acctId = id;
    }
```

Solution

```
List<Account> sorted
  = accounts.stream()
            .sorted(Comparator.comparing((Account a) -> a.getBalance())
                    .thenComparing(a -> a.getOwnerName()))
            .collect(Collectors.toList());
```

Main Point 3

- The **`comparing`** and **`thenComparing`** methods are examples of the new declarative style that can be used in working with Streams in Java 8.
  - This style of programming makes it possible to obtain results simply by declaring what is needed, rather than specifying in detail how to obtain the results.
- In a similar way, as awareness becomes more and more familiar with its unbounded transcendental value, we are able to achieve goals with less effort
  - Because transcending engages the deeper levels of nature's functioning, winning support for our individual intentions.